

Assignment

Ques: What is ENUM in Java. Explain with Example?

Sol: An **Enum** (short for *enumeration*) is a special class that represents a fixed set of constants. Use it when a variable can only take one value from a known, limited list, like days of the week, directions, or order status.

Why use Enums?

- Type-safe: you can't assign an invalid value
- More readable than plain `int` or `String` constants
- Work well with `switch`
- Can have fields, constructors, and methods

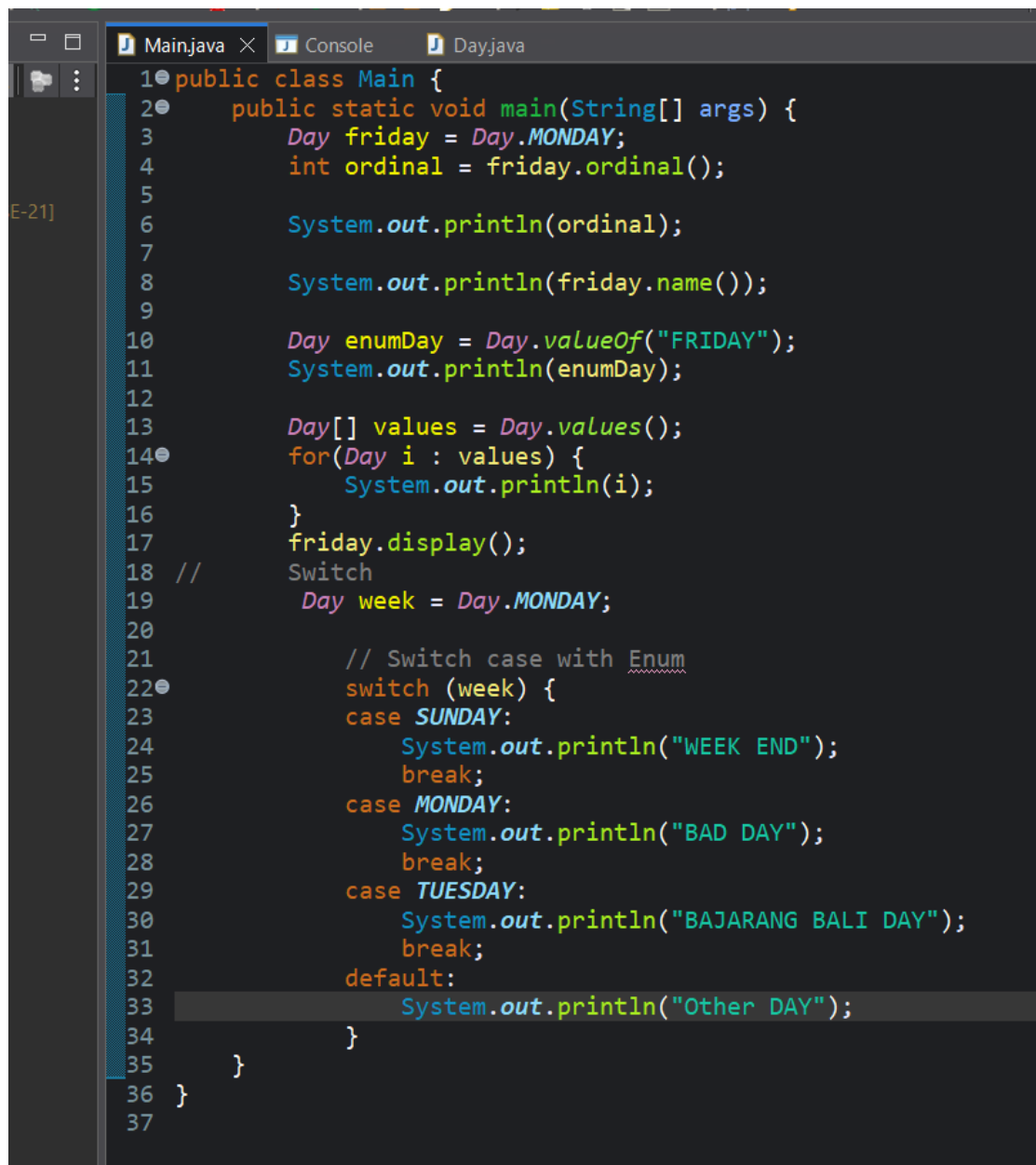
Key points to remember

Feature	Detail
Keyword	<code>enum</code>
Constants	Written in UPPERCASE by convention
Inheritance	An enum implicitly extends <code>java.lang.Enum</code> , so it can't extend another class (but it can implement interfaces)
Instantiation	You cannot create enum objects with <code>new</code>
Comparison	Use <code>==</code> to compare enum values
Useful methods	<code>values()</code> , <code>valueOf()</code> , <code>ordinal()</code> , <code>name()</code>

```
1
2 public enum Day {
3     SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY;
4
5     public void display() {
6         System.out.println("Today is " + this.name());
7     }
8 }
9
```

```
Main.java x Console Day.java
1 public class Main {
2     public static void main(String[] args) {
3         Day friday = Day.MONDAY;
4         int ordinal = friday.ordinal();
5
6         System.out.println(ordinal);
7
8         System.out.println(friday.name());
9
10        Day enumDay = Day.valueOf("FRIDAY");
11        System.out.println(enumDay);
12
13        Day[] values = Day.values();
14        for(Day i : values) {
15            System.out.println(i);
16        }
17        friday.display();
18    }
19 }
20
```

```
terminated> Main (9) [Java Application] C:\Users\ajeet\p2\pool\plugins\org.e
1
MONDAY
FRIDAY
SUNDAY
MONDAY
TUESDAY
WEDNESDAY
THURSDAY
FRIDAY
SATURDAY
Today is MONDAY
```



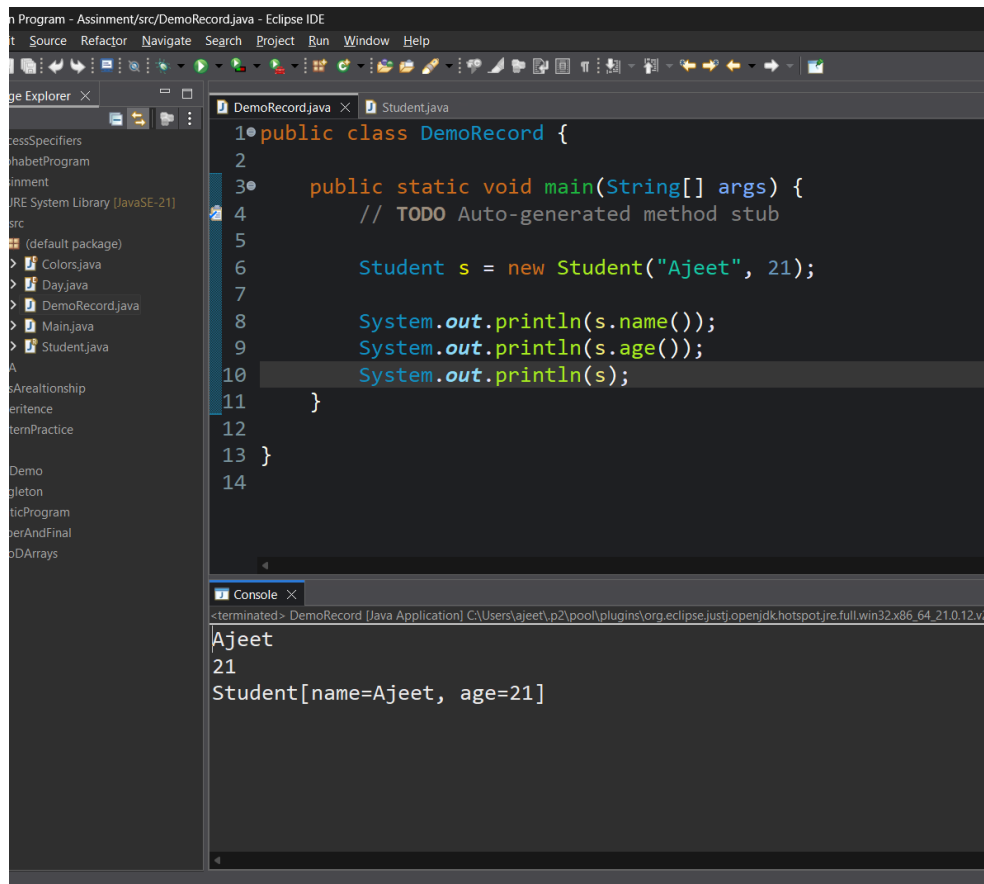
```
1 public class Main {
2     public static void main(String[] args) {
3         Day friday = Day.MONDAY;
4         int ordinal = friday.ordinal();
5
6         System.out.println(ordinal);
7
8         System.out.println(friday.name());
9
10        Day enumDay = Day.valueOf("FRIDAY");
11        System.out.println(enumDay);
12
13        Day[] values = Day.values();
14        for(Day i : values) {
15            System.out.println(i);
16        }
17        friday.display();
18        // Switch
19        Day week = Day.MONDAY;
20
21        // Switch case with Enum
22        switch (week) {
23            case SUNDAY:
24                System.out.println("WEEK END");
25                break;
26            case MONDAY:
27                System.out.println("BAD DAY");
28                break;
29            case TUESDAY:
30                System.out.println("BAJARANG BALI DAY");
31                break;
32            default:
33                System.out.println("Other DAY");
34        }
35    }
36 }
37
```

QUES 2: What is Record in Java. Explain with Example?

Solutions:

A **record** is a special kind of class, introduced in Java 14 as a preview feature and made official in **Java 16**, designed to hold **immutable data**. It removes the boilerplate you normally write for simple data-carrier classes.

Normally, a class that just stores data needs fields, a constructor, getters, equals(), hashCode(), and toString(). With a record, you declare only the fields and Java generates all of that for you.

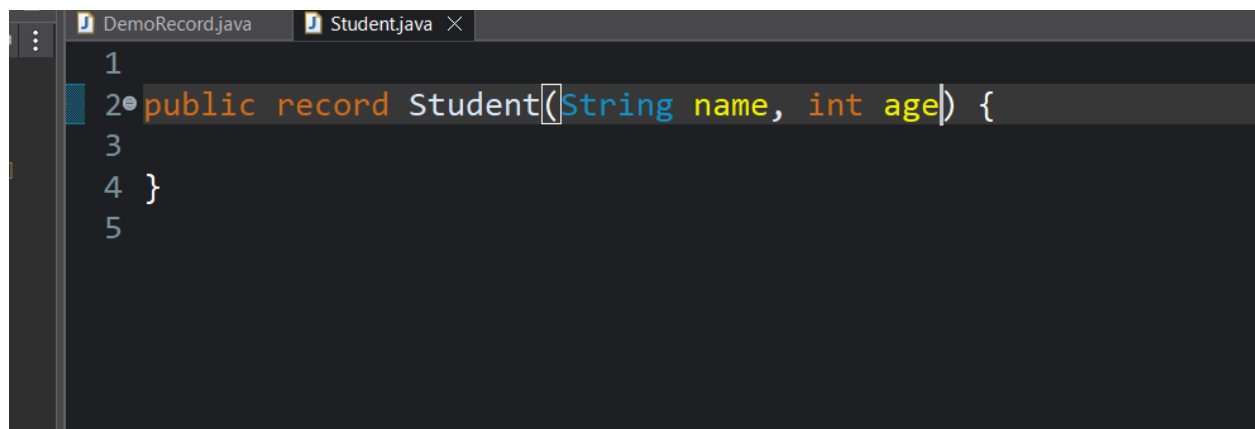


The screenshot shows the Eclipse IDE with a project named 'Assinment'. The package explorer on the left shows the source folder 'src' containing files like Colors.java, Day.java, DemoRecord.java, Main.java, and Student.java. The main editor displays 'DemoRecord.java' with the following code:

```
1 public class DemoRecord {
2
3     public static void main(String[] args) {
4         // TODO Auto-generated method stub
5
6         Student s = new Student("Ajeet", 21);
7
8         System.out.println(s.name());
9         System.out.println(s.age());
10        System.out.println(s);
11    }
12
13 }
14
```

The console window at the bottom shows the output of the program:

```
<terminated> DemoRecord [Java Application] C:\Users\ajeet\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.12.v20
Ajeet
21
Student[name=Ajeet, age=21]
```



The screenshot shows the Eclipse IDE with the 'Student.java' file open. It defines a record named 'Student' with two fields: 'name' of type 'String' and 'age' of type 'int'.

```
1
2 public record Student(String name, int age) {
3
4 }
5
```

Without Using Reord

```
avigate Search Project Run Window Help
1 import java.util.Objects;
2
3 public class StudDetails {
4     String name;
5     int age;
6
7     public StudDetails(String name, int age) {
8         super();
9         this.name = name;
10        this.age = age;
11    }
12
13    public String getName() {
14        return name;
15    }
16    public int getAge() {
17        return age;
18    }
19
20    @Override
21    public int hashCode() {
22        return Objects.hash(Integer.valueOf(age), name);
23    }
24
25    @Override
26    public boolean equals(Object obj) {
27        if (this == obj)
28            return true;
29        if (obj == null)
30            return false;
31        if (getClass() != obj.getClass())
32            return false;
33        StudDetails other = (StudDetails) obj;
34        return age == other.age && Objects.equals(name, other.name);
35    }
36
37    @Override
38    public String toString() {
39        return "StudDetails [name=" + name + ", age=" + age + ", getName()=" + getName() + ", getAge()=" + getAge()
40            + ", hashCode()=" + hashCode() + ", getClass()=" + getClass() + ", toString()=" + super.toString()

```

ssinment/src/DemoRecord.java - Eclipse IDE

Refactor Navigate Search Project Run Window Help

DemoRecord.java Student.java StudDetails.java

```
1 public class DemoRecord {
2
3     public static void main(String[] args) {
4         // TODO Auto-generated method stub
5
6         Student s = new Student("Ajeet", 21);
7
8         System.out.println(s.name());
9         System.out.println(s.age());
10        System.out.println(s);
11
12        // Without record
13        StudDetails st = new StudDetails("Sumit", 1);
14        System.out.println(st.age);
15        System.out.println(st.name);
16        System.out.println(st);
17
18        System.out.println(st.equals(s));
19    }
20
21 }
22
```

Result.

Console

terminated> DemoRecord [Java Application] C:\Users\ajeet\p2\poo\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.12.v20260826-1216\jre\bin\javaw.exe (Oct 4, 2026, 12:27:49 AM - 12:27:49 AM elapsed 0:00:00) [pid: 916]

```
Ajeet
21
Student[name=Ajeet, age=21]
1
Sumit
StudDetails [name=Sumit, age=1, getName()=Sumit, getAge()=1, hashCode()=80246902, getClass()=class StudDetails, toString()=StudDetails@4c87876]
false
```

Your code	In the record
-----------	---------------

Fields name, age	Declared in the header, automatically private final
Constructor	Canonical constructor, generated
getName(), getAge()	name(), age()
hashCode()	Generated, based on all components
equals()	Generated, compares all components
toString()	Generated: StudDetails[name=Ajeet, age=1]

What Java generates automatically

- A private final field for each component
- A canonical constructor
- Accessor methods (name(), age())
- equals(), hashCode(), and toString()

Compact constructor (validation)

You can add validation without rewriting the whole constructor:

The screenshot shows the Eclipse IDE interface. The 'Project Explorer' on the left lists several files, including 'Student.java'. The main editor window displays the code for 'Student.java', which is a record with a compact constructor. The code is as follows:

```

1 record Student(String name, int age) {
2     Student {
3         if (age < 0) {
4             throw new IllegalArgumentException("Age cannot be negative");
5         }
6     }
7 }

```

Adding methods and static members

Records can have extra methods, static fields, and static methods:

```

1 record Rectangle(double length, double width) {
2
3     // custom method
4     double area() {
5         return length * width;
6     }
7
8     // static method
9     static Rectangle square(double side) {
10        return new Rectangle(side, side);
11    }
12 }
13

```

Rules and limitations

Point	Detail
Immutability	Fields are <code>final</code> ; you can't change them after creation
Inheritance	A record cannot extend another class (it implicitly extends <code>java.lang.Record</code>)
Subclassing	A record is implicitly <code>final</code> , so nothing can extend it
Interfaces	A record can implement interfaces
Instance fields	You cannot declare extra instance fields beyond the components
Setters	None are generated, since the data is immutable

Record vs. normal class

	Record	Normal class
Purpose	Immutable data holder	General purpose
Boilerplate	Almost none	A lot
Fields mutable?	No	Yes (if you want)
equals/toString	Auto-generated	You write them

When to use records

Use them for DTOs, API responses, coordinates, return values bundling multiple items, and map keys. Avoid them when you need mutable state or inheritance, such as JPA entities.

I can also show how records work with enums or streams if that would help.

Ques 3: What is Module Import in Java. Explain with Example?

Solutions:

Module Imports in Java

"Module imports" can mean two related things, so here are both.

1. Module import declarations (`import module`)

This is a newer feature. It was a preview in Java 23 and 24 and became **final in Java 25**. It lets you import **all the packages exported by a module** with one line, instead of writing many separate imports.

Without a module import:

```
java
import java.util.List;
import java.util.ArrayList;
import java.util.Map;
import java.util.HashMap;
import java.util.stream.Collectors;
import java.io.File;
```

inment/src/Modullmp.java - Eclipse IDE

factor Navigate Search Project Run Window Help

DemoRecord.java Student.java StudDetails.java Rectangle.java Modullmp.java X

```
1 import module java.base;
2
3 import module java.base;
4 import module java.sql;
5
6 import java.util.Date;
7 public class ModulImp {
8     public static void main(String[] args) {
9         List<String> names = new ArrayList<>();
10        names.add("Rahul");
11        names.add("Priya");
12
13        Map<String, Integer> ages = new HashMap<>();
14        ages.put("Rahul", 20);
15
16        String joined = names.stream().collect(Collectors.joining(", "));
17        System.out.println(joined);
18        System.out.println(ages);
19    }
20 }
```

Console X

<terminated> ModulImp [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (Oct 4, 2026, 1:42:30 AM – 1:42:30 AM elapsed 0

Rahul, Priya
{Rahul=20}